10.2.1 Schemas and validation

One of the first lessons ACME learned was that policies without schemas quickly become inconsistent. Developers on the Customer Collaboration platform had written a policy using the action `doc:modify`, while another team used `doc:edit`. Both worked in their own code, but when policies were combined, problems arose.

Introducing Cedar schemas resolved the issue. Every repository included a schema that defined valid principal types, resource types, and actions. From then on, pull requests couldn’t be merged unless policies were type-checked against the schema.

In one memorable case, an engineer attempted to write a condition to check the `department` attribute:

```
permit(
    principal,
    action == Action::"deploy",
    resource in DeploymentTool
)
when { principal.department == "Engineering" };
```

The schema flagged this as invalid because `department` wasn’t a modeled attribute of principals. Rather than silently enforce a broken rule, the error prompted a design discussion with HR about how to represent organizational data. The corrected version relied on roles defined in the entity store:

```
permit(
    principal,
    action == Action::"deploy",
    resource in DeploymentTool
)
when {
    principal in Role::"engineering-manager" ||
    principal in Role::"senior-engineer"
};
```

Validation not only maintained policy consistency; it also revealed gaps in ACME’s data model and encouraged the company to clearly define where